

# Knot Cutting & Region Coloring Algorithm（现行实现）

## 1 总览

Torus Knot 切割把一条解析曲线  $v = k u$ （在 UV 参数域内实际有多条分支  $v = k u + 2\pi n$ ）投影到网格表面，沿曲线把与之相交的三角形/四边形切开，并按曲线在 UV 平面划分的“条带”为每个面分配区域颜色。

关键事实（纠正旧文档）：

1. 一条 knot 曲线在  $[0, 2\pi] \times [0, 2\pi]$  的参数方域内有  $k$  条分支（每  $2\pi$  一条）。必须切开面跨越的全部分支，旧实现只切最低分支（ $n_{lo}$ ）会大量漏切，导致实际切割边与渲染的完整曲线不一致。
2. 区域编号不使用符号位编码  $pu = s1 \mid (s2 < 1)$ ，而是用  $(v - k*u).div\_euclid(2\pi)$  得到每个 knot 的“条带编号”，再用 BTreeSet 对所有面的条带向量去重，得到连续的 UV 平面区域颜色编号。

## 2 网格：展开 UV 域

顶点在  $u = 0$  与  $u = 2\pi$  处是独立顶点（由 `generate_unfolded_quad_mesh` 生成  $(res\_u + 1) \times (res\_v + 1)$  顶点），没有面跨过周期边界。所有顶点 UV 坐标  $\in [0, 2\pi]$ 。

`clamp_uv` 把交点夹到  $[0, 2\pi]$  而不是模  $2\pi$  包裹，从而保留  $u = 0$  与  $u = 2\pi$  作为两个独立边界。

## 3 相交检测：intersect\_face\_knot\_branch

给定面顶点  $\{(u_i, v_i)\}$ 、knot 参数  $k$  与分支号 `branch`：

### Step 1：归一化并展开 UV

对第一个顶点取参考  $(u_0, v_0)$ ，把每个 UV 归一化到  $[0, 2\pi]$ ：

$$\bar{u}_i = \text{normalize\_uv}(u_i, \min_u, \max_u), \quad \bar{v}_i = \text{normalize\_uv}(v_i, \min_v, \max_v)$$

再相对参考展开（保证  $\bar{u}_i - \bar{u}_0 \in [-\pi, \pi]$ ，处理接缝）：

$$a_i = \text{unwrap}(\bar{u}_i, \bar{u}_0), \quad b_i = \text{unwrap}(\bar{v}_i, \bar{v}_0)$$

其中 `unwrap(x, r)` 通过  $\pm 2\pi$  调整使  $|x - r| \leq \pi$ 。

## Step 2: 计算 $\phi$ 与目标分支

$$\phi_i = b_i - k a_i$$

目标分支对应的常数为:

$$\text{target} = 2\pi \cdot \text{branch}$$

## Step 3: 顶点分类 (Side)

$$\text{side}_i = \begin{cases} \text{OnLine} & |\phi_i - \text{target}| < \text{MERGE\_EPS} \\ \text{Less} & \phi_i - \text{target} < 0 \\ \text{Greater} & \phi_i - \text{target} > 0 \end{cases}$$

其中  $\text{MERGE\_EPS} = 10^{-4}$  (用于“顶点在 knot 线上”的判定, 比  $\text{EPSILON} = 10^{-8}$  宽松, 因为相邻边分裂产生的顶点经 `unwrap+wrap` 往返后  $\phi$  可能略超  $10^{-8}$ )。

## Step 4: 求边交点

对每条边  $(i, i+1)$ , 若两端 side 不同或涉及 OnLine:

$$t = \frac{-(\phi_i - \text{target})}{(\phi_{i+1} - \text{target}) - (\phi_i - \text{target})}, \quad \text{夹到}[0, 1]$$

过滤: 起点 OnLine 且  $t < \text{MERGE\_EPS}$  跳过; 终点 OnLine 且  $t > 1 - \text{MERGE\_EPS}$  跳过。新顶点 UV 用 `clamp_uv` 夹到  $[0, 2\pi]$ 。

## 4 面跨越的分支范围: `knot_branch_range`

对整张面计算  $\phi \in [\phi_{\min}, \phi_{\max}]$ , 则覆盖的分支为:

$$n_{\text{lo}} = \left\lceil \frac{\phi_{\min}}{2\pi} \right\rceil, \quad n_{\text{hi}} = \left\lfloor \frac{\phi_{\max}}{2\pi} \right\rfloor$$

若  $n_{\text{lo}} > n_{\text{hi}}$ : 面不跨任何分支, 返回无交点。

## 5 面切割: `cut_face_local_knot`

这是与旧文档“只处理第一个分支”最本质的区别:

```
1 let (n_lo, n_hi) = knot_branch_range(&get_face_uvs(mesh, face), k, uv_range);
2 if n_lo > n_hi { return false; }
3 let mut cut_any = false;
4 for branch in n_lo..=n_hi { // 遍历 ALL 分支
5     let face_verts = get_face_uvs(mesh, face);
6     let Some(isect) = intersect_face_knot_branch(&face_verts, k, branch, uv_range)
7     else { continue; };
8     // 相交的非三角形面先局部三角化, 再按交点切分
9     if !isect.edge_points.is_empty() {
10         let hes = mesh.face_half_edges(face);
```

```

10         if hes.len() > 3 { triangulate_face(mesh, face); }
11     }
12     let face_verts = get_face_uvs(mesh, face);
13     if let Some(isect) = intersect_face_knot_branch(&face_verts, k, branch,
14         uv_range) {
15         cut_face_by_intersection(mesh, face, &isect);
16         cut_any = true;
17     }
18 }
cut_any

```

也就是说，一个面若同时跨越多条 knot 分支，会被逐分支依次切开——这正是 CHANGELOG 中 “multi-branch knot fix” 所修复的行为。

## 6 逐线切割与修复：cut\_mesh\_by\_knot\_line

对每条 knot 参数  $k$ ：

1. 单遍扫描所有面，调用 `cut_face_local_knot` 切割。
2. **修复遍**：单遍可能漏掉“切割线恰好穿过顶点”的相邻面。重复最多 5 遍：对每个有效面计算  $\phi = v - k u$  在各分支上的 min / max，若某分支 target 两侧分别存在顶点（has\_below 且 has\_above，容差  $10^{-4}$ ），说明该面仍骑跨该分支，则再次调用 `cut_face_local_knot`。修复遍无新切割即停止。

## 7 区域着色：assign\_multi\_knot\_patch\_indices

为每个有效面计算一个“条带向量”，去重后映射为连续颜色编号。

### Step 1: 单 knot 条带函数

对每个 knot  $k$ ，条带编号定义为：

$$\text{band}(u, v, k) = (v - k u) \bmod 2\pi \in \mathbb{Z}$$

（注意：div\_euclid 对整数  $k$  在  $(u, v) \rightarrow (u + 2\pi, v + 2\pi)$  下不变，故可直接用原始 UV 值，无需先归一化。）

### Step 2: 选最无歧义的顶点

对每个面，枚举其所有顶点，计算每个顶点到各 knot 线最近分支的有符号距离，取“离所有 knot 线最远”的顶点  $(u, v)$  作为分类依据。混合策略：

- 若该顶点明显远离所有 knot 线（最小距离  $> 0.1$ ），直接用其  $(u, v)$  分类——稳健且精确；
- 否则（面靠近 knot 交点）改用 `circular_centroid`（先相对首顶点展开、再取圆均值）以获得更稳定的方向信号。

### Step 3: 构造条带向量并去重

$$\text{bands} = [\text{band}(u, v, k_1), \text{band}(u, v, k_2), \dots]$$

把所有面的 `bands` 插入 `BTreeSet<Vec<isize>>` 去重，得到全部互异的条带向量；再为每个互异向量分配一个连续编号  $c \in \{0, 1, \dots\}$ 。面的 `patch_index` 即设为 `(pair_index[bands], 0)`。

### UV 平面区域语义

knot 曲线  $v = ku + 2\pi n$  的  $k$  条分支把 UV 平面分成  $k+1$  条带，每条带一个区域颜色 (ByRegion 着色需求)。环面拓扑上单条闭曲线不分割环面（所有带经包裹连通），但 UV 展开平面的视觉区域应独立着色——即用户可见的“区域”。

## 8 着色查找与可见性: `rebuild_render_state`

渲染时每个面的颜色索引为 `pu = patch_index.0`，颜色取自 `color_scheme::generate_patch_colors(max_patch_index, scheme)`。`patch_visible` 在每次 `reapply_cuts` 时重置为 `[true; max_pu+1]`，可在 Shader 标签页逐补片显隐。

## 9 已知边界与注意事项

- **多分支切割开销:**粗网格 + 大  $k$  时一个面可能跨  $k$  条分支，全部切开会显著增加面数；`cut_mesh_by_grid` 中另有 `MAX_FACES=150000` 硬上限防止指数增长（knot 切割本身无此上限，依赖修复遍最多 5 次收敛）。
- **顶点恰在 knot 线上:** 相邻两边交点被  $t \approx 0/1$  过滤后可能得到  $(0, 1)$  情形 (do nothing)，面不切分——由于仅一个顶点接触曲线，面整体位于一侧，行为正确。
- **质心 vs 几何:** 切割后每个面应整体落在一条带内；若某面本应被切却漏切，其质心可能落入某侧而部分几何落入另一侧，产生区域边界“锯齿”——多分支切割修复遍正是为消除此类漏切。